# Mechatron Computational Architecture

*A Universal Computational Architecture Unifying Independent Functional Units, a Single Common Representation, Systematic Multi-Mechanism Reasoning, Dynamic Lead Generation, Interaction and Cross-Contribution, and Verification*

**Ashok Radhakrishnan, M.E.**

**Mechatron Computational Architecture**
**Copyright © 2026 Ashok Radhakrishnan, M.E.**
**All rights reserved.**

This publication can be cited, quoted, and referenced in academic, scientific, technical, and scholarly works with appropriate attribution.

The concepts presented in this work originate from the author's research and development.

**Software Tool Usage**
ChatGPT

# Table of Contents

**Part I – Architecture**

1. **Abstract**
2. **Engineering Foundation**
3. **Operational Principles**
    - 3.1 Operational Principles Purpose
    - 3.2 Required Operational Properties
    - 3.3 Excluded Operational Properties
    - 3.4 Computational Error Recognition
    - 3.5 Operational Objective
    - 3.6 Systematic Computation
    - 3.7 Unknown Information Handling
    - 3.8 Objective-Oriented Computation
    - 3.9 Modular Architecture
4. **Architectural Principles**
    - 4.1 Separation of Responsibilities
    - 4.2 Architectural Symmetry
    - 4.3 Common Representation
    - 4.4 Common Representation Engineering Relationship
    - 4.5 Extensibility
    - 4.6 Stable Architecture
5. **Computational Workflow**
6. **Memory Architecture**
    - 6.1 Memory Architecture Purpose
    - 6.2 Working Memory
    - 6.3 Long-Term Storage
    - 6.4 User-Controlled Information
    - 6.5 Memory Architecture Objective
7. **Heterogeneous Inputs**
8. **Input Sensor Processing Units**
    - 8.1 ISPU Architectural Rule
9. **Input Integration Unit (IIU)**
10. **Common Representation**
    - 10.1 Engineering Representation
    - 10.2 Physical Communication Medium
    - 10.3 Common Representation Continuity
11. **Central Computational Unit (CCU)**
    - 11.1 CCU Purpose
    - 11.2 Principle of Computational Reasoning
    - 11.3 CCU Computational Reasoning Process
    - 11.4 Computational Reasoning Mechanisms
    - 11.5 Interaction Pattern Learning
    - 11.6 CCU Core Responsibilities
    - 11.7 CCU Computational Objective
12. **Output Integration Unit (OIU)**
    - 12.1 OIU Purpose
    - 12.2 Output Integration
    - 12.3 Multiple Output Representations
    - 12.4 Output Coordination
    - 12.5 OIU Core Responsibilities
    - 12.6 OIU Output Objective
13. **Output Sensor Processing Units**
    - 13.1 OSPU Purpose

- 13.2 OSPU Architectural Rule
- 13.3 OSPU Output Processing
- 13.4 Standardised Architectural Interface
- 13.5 OSPU Core Responsibilities
- 13.6 OSPU Output Objective

14. **Heterogeneous Outputs**
    - 14.1 Heterogeneous Outputs Purpose
    - 14.2 Output Technologies
    - 14.3 Heterogeneous Outputs Objective
15. **Output Verification**
    - 15.1 Output Verification Purpose
    - 15.2 Output Verification Process
    - 15.3 Output Verification Outcome
    - 15.4 Output Verification Objective
16. **Computational Workflow Iteration**
    - 16.1 Workflow Iteration Purpose
    - 16.2 Iterative Computation
    - 16.3 Evidence-Based Iteration
    - 16.4 Workflow Continuity
    - 16.5 Workflow Iteration Objective
17. **Continuous Computational Improvement**
    - 17.1 Continuous Improvement Purpose
    - 17.2 Principles of Continuous Improvement
    - 17.3 Scope of Improvement
    - 17.4 Verified Improvement
    - 17.5 Continuous Improvement Objective
18. **Technology Independence**
19. **Architectural Independence**
20. **Summary**

**Part II – Common Representation**

1. **Purpose**
2. **Design Principles**
3. **Common Representation Lifecycle**
4. **Representation Invariance**
5. **Common Representation Structure**
    - 5.1 Purpose
    - 5.2 Standardised Representation
    - 5.3 Computational Content
    - 5.4 Standardised Metadata
    - 5.5 Technology Independence
    - 5.6 Engineering Representation
    - 5.7 Physical Communication Medium
    - 5.8 Representation Selection Principles
    - 5.9 Extensibility
6. **Common Representation Operations**
    - 6.1 Purpose
    - 6.2 Conversion and Integration (IIU)
    - 6.3 CCU Computational Reasoning Process
    - 6.4 Computational Content Update and Continued Computation
    - 6.5 Transmission
    - 6.6 Consumption (OIU)
7. **Representation Integrity**
    - 7.1 Standardisation
    - 7.2 Technology Independence

    - 7.3 No Representation Conversion within the CCU
    - 7.4 Conversion to OSPU-Specific Output Representations Only within the OIU
8. **Architectural Hierarchy**
9. **Summary**

**Part III – Mathematical Model**

1. **Input Sensor Processing Unit (ISPU)**
2. **Input Integration Unit (IIU)**
3. **Central Computational Unit (CCU)**
4. **Output Integration Unit (OIU)**
5. **Output Sensor Processing Unit (OSPU)**
6. **Output Verification and Iteration**
7. **Complete Computational Workflow Model**

**Part IV – Algorithms**

1. **Input Sensor Processing Unit (ISPU)**
2. **Input Integration Unit (IIU)**
3. **Central Computational Unit (CCU)**
4. **Output Integration Unit (OIU)**
5. **Output Sensor Processing Unit (OSPU)**
6. **Output Verification and Iteration**
7. **Complete Algorithmic Flow**

# Part I
# Architecture

**1. Abstract**
The Mechatron Computational Architecture (MCA) is a technology-independent architectural specification for computational machines, defining the organisation, responsibilities, interfaces, relationships, and workflow of their computational components. Heterogeneous information is processed through Input Sensor Processing Units (ISPUs). The Input Integration Unit (IIU) converts each processed input into a Common Representation (CR) and integrates the resulting representations into a single CR. The Central Computational Unit (CCU) determines the computational meaning of the CR and performs systematic reasoning through multiple mechanisms operating in parallel, simultaneously, and concurrently. These mechanisms dynamically generate computational leads, interact, and cross-contribute, allowing information, intermediate results, deductions, constraints, evaluations, patterns, and other contributions from one mechanism to influence others, including mechanisms whose relevance was not initially determined. Their contributions are evaluated to determine a sufficiently supported computational solution. If no sufficiently supported solution emerges, the CCU assesses information sufficiency and acquires additional information only when required. If the information is already available but reasoning fails, the reasoning process requires diagnosis or troubleshooting. The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome. The CR itself remains unchanged while its computational content is updated. The Output Integration Unit (OIU) converts the updated computational content into OSPU-specific Output Representations, which the corresponding Output Sensor Processing Units (OSPUs) process to produce outputs for associated sensors, actuators, or devices. These outputs are verified against the computational objective. MCA provides a unified architectural framework for heterogeneous information integration, multi-mechanism computational reasoning, dynamic interaction and cross-contribution, information sufficiency assessment, and output verification.

## 2. Engineering Foundation
The Mechatron Computational Architecture is founded upon engineering principles. Its objective is systematic computation through a stable, technology-independent computational architecture.

It defines a computational architecture capable of:
- systematic information processing
- objective-oriented computation
- Common Representation–based computation
- technology-independent operation
- verification of computational outcomes
- continuous computational improvement

By separating architectural responsibilities from engineering implementations, the architecture provides a stable computational specification that remains applicable across diverse engineering domains while allowing implementation technologies to evolve without modifying the architecture.

## 3. Operational Principles
### 3.1 Operational Principles Purpose
The Operation of a Mechatron Machine is governed by engineering principles and defined operational properties. Operational properties represent the externally observable characteristics exhibited by the machine while interacting with users, systems, and environments. The objective of these operational principles is to promote reliable, predictable, transparent, and systematic computational operation.

### 3.2 Required Operational Properties
A Mechatron Machine shall exhibit operational properties appropriate to an engineered computational system, including:
- transparency
- consistency

- predictability
- traceability
- safety
- computational accuracy
- computational integrity
- procedural compliance

These properties support objective, evidence-based computation and predictable system operation.

**3.3 Excluded Operational Properties**

The architecture excludes operational properties that reduce computational reliability, including:

- non-computational response states
- non-objective preference criteria
- subjective evaluation criteria
- biased decision criteria
- non-objective decision criteria
- self-referential operation
- adversarial response operation
- deceptive interaction operation

These properties are excluded because they are inconsistent with objective, systematic, and reliable computational operation.

**3.4 Computational Error Recognition**

A Mechatron Machine may recognise and report computational errors when they occur. Recognition of computational error represents objective identification of computational inaccuracy based on defined computational criteria.

**3.5 Operational Objective**

The objective of the Operational Principles is to ensure that every Mechatron Machine operates consistently with reliable, objective, systematic, and evidence-based computation.

**3.6 Systematic Computation**

Every computational result shall be produced through systematic computation. Computational results shall not be produced through arbitrary selection, unsupported inference, or assumption.

**3.7 Unknown Information Handling**

Unknown information shall never be replaced by assumptions. When additional information is required to satisfy the current computational objective, the architecture shall acquire that information before computation continues. If the information is already available but reasoning fails, the reasoning process requires diagnosis or troubleshooting. Information insufficiency and reasoning failure are distinct conditions.

**3.8 Objective-Oriented Computation**

Every computational workflow shall operate toward a defined computational objective. The computational objective determines when computation begins, continues, and terminates.

**3.9 Modular Architecture**

The architecture consists of independent computational components. Each component performs a defined computational responsibility and communicates through defined architectural interfaces.

## 4. Architectural Principles

The Mechatron Computational Architecture is defined by the following architectural principles.

**4.1 Separation of Responsibilities**

Each architectural component shall perform one primary computational responsibility.

### 4.2 Architectural Symmetry

The architecture maintains a symmetrical computational organisation between information entering and leaving the computational system.

### 4.3 Common Representation

All internal computational processing shall operate on a single Common Representation (CR).

- the Common Representation is an architectural abstraction
- its engineering realisation is implementation-dependent
- one Common Representation exists from IIU → CCU → OIU
- the engineering representation remains unchanged during internal computation
- physical communication media are implementation choices

The Input Integration Unit (IIU) converts each processed input received from the Input Sensor Processing Units (ISPUs) into the Common Representation (CR) and integrates the resulting Common Representation inputs into a single Common Representation. The Common Representation is then processed by the Central Computational Unit (CCU) and consumed by the Output Integration Unit (OIU).

### 4.4 Common Representation Engineering Relationship

The Common Representation is an architectural representation of integrated computational information. Its engineering realisation is defined through an Engineering Representation selected according to the engineering purpose and implementation requirements. The Engineering Representation is the engineering realisation used to represent, process, store, and communicate the Common Representation. Binary Representation is the preferred Engineering Representation; however, alternative Engineering Representations may be selected according to engineering requirements. The Physical Communication Medium provides the physical mechanism for transporting the selected Engineering Representation between architectural components. Optical communication is the preferred Physical Communication Medium; however, alternative physical communication medium may be selected according to engineering requirements. The Engineering Representation and Physical Communication Medium do not alter the architectural definition of the Common Representation. The Common Representation remains consistent from the Input Integration Unit (IIU), through the Central Computational Unit (CCU), to the Output Integration Unit (OIU).

### 4.5 Extensibility

Additional sensing technologies, computational methods, output technologies, and engineering implementations may be incorporated without modifying the architectural structure.

### 4.6 Stable Architecture

The architecture defines computational organisation and functional responsibilities. Implementations may evolve. The architecture remains unchanged.

## 5. Computational Workflow

The Mechatron Computational Architecture defines a technology-independent computational workflow governing the execution of every computational objective. All computational objectives follow the same architectural workflow across different implementation technologies, computational methods, sensing technologies, input technologies, and output technologies. A single Common Representation is maintained across the IIU, CCU, and OIU. Its engineering representation and the physical communication medium are implementation-dependent but remain consistent throughout the internal computational workflow.

## The computational workflow consists of:

**Heterogeneous Input Sensor Types**

↓

**Input Sensor Processing Units (ISPUs)**

Processes information from its associated input sensor type.

↓

**Input Integration Unit (IIU)**

Converts each processed input into the Common Representation (CR) and integrates the resulting Common Representation inputs into a single Common Representation.

↓

**Single Common Representation (CR)**

↓

**Central Computational Unit (CCU)**

- Reads the Common Representation and determines the computational meaning of the information represented within it.
- Performs systematic computational reasoning using the Common Representation.
- Participating reasoning mechanisms operate in parallel, simultaneously, and concurrently and may generate computational leads and dynamically interact through computational cross-contribution.
- Information, intermediate results, deductions, constraints, evaluations, patterns, and other computational contributions produced by one reasoning mechanism may generate computational leads that contribute to and influence the computation performed by another reasoning mechanism, including a mechanism whose relevance was not initially determined.
- If a sufficiently supported computational solution has not emerged through this dynamic reasoning, the CCU assesses information sufficiency and, when required, acquires additional information before computation continues using the same Common Representation with updated computational content.
- If information is available but reasoning fails, diagnosis or troubleshooting is needed.
- Once a sufficiently supported computational solution emerges, the CCU evaluates and compares the computational solution or solutions, selects the solution that satisfies the computational objective, and updates the computational content of the same Common Representation.
- The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome.

↓

**Same Common Representation (CR) containing updated computational content**

↓

**Output Integration Unit (OIU)**

Converts the computational content of the Common Representation into the required OSPU-specific Output Representations, including associated metadata, sequencing, coordination, synchronisation, prioritisation, consolidation, and routing information.

↓

**Corresponding Output Sensor Processing Unit(s) (OSPUs)**

Processes the output representation received for its associated output sensor type.

↓

**Selected Corresponding Output Sensor / Actuator / Device**

Receives the processed output for execution.

↓

**Output Verification**

Verification determines whether the computational objective has been achieved.

- **If the computational objective has been achieved, the computational workflow terminates.**
- **If the computational objective has not been achieved, the architecture acquires additional information and continues systematic computation.**

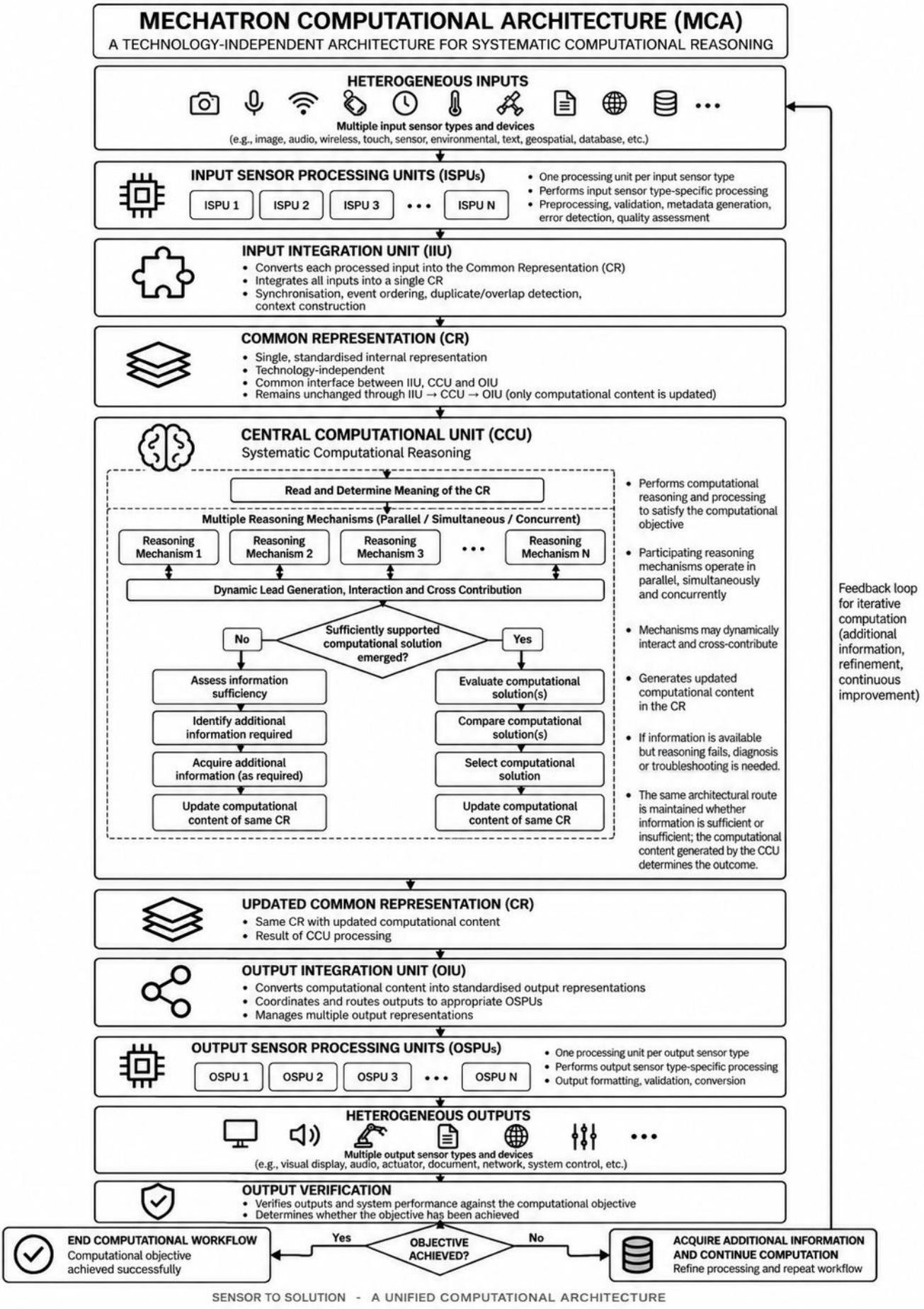



**Figure 1. Mechatron Computational Architecture**

# 6. Memory Architecture

### 6.1 Memory Architecture Purpose

The Memory Architecture provides the information storage capabilities required to support systematic computation within the Mechatron Computational Architecture. The architecture separates temporary computational information from persistent information to ensure efficient computational processing while preserving computational integrity.

The Memory Architecture consists of two primary components:

- Working Memory
- Long-Term Storage

The architecture defines the responsibilities of memory independently of storage technologies, database systems, file systems, hardware platforms, and engineering implementations.

### 6.2 Working Memory

Working Memory temporarily stores information required during an active computational workflow.

Typical contents include:

- the current computational objective
- the current computational context
- Common Representation containing the computational content of the current computational state
- intermediate computational results
- temporary input information
- temporary output information
- temporary computational state

Information shall remain in Working Memory only for the duration required to satisfy the computational objective. Temporary information shall be discarded when it is no longer required.

### 6.3 Long-Term Storage

Long-Term Storage preserves information intended for continued use beyond the current computational workflow.

Examples include, but are not limited to:

- computational procedures
- engineering rules
- interaction models
- user-authorised information
- user documents
- user media
- system configuration
- domain knowledge
- other persistent computational information

Long-Term Storage supports systematic computation without relying upon memorisation of previous questions, conversations, or answers as a computational strategy.

### 6.4 User-Controlled Information

The Mechatron Machine operates as a user-controlled computational system. Information designated as user-owned remains under the control of the user. Operations affecting persistent user information, including deletion or permanent modification, shall require user authorisation unless explicitly configured otherwise.

### 6.5 Memory Architecture Objective

The objective of the Memory Architecture is to provide reliable storage for temporary and persistent computational information while supporting systematic, evidence-based computation independently of implementation technologies. The Memory Architecture defines storage responsibilities independently of storage implementation.

## 7. Heterogeneous Inputs

Heterogeneous Inputs constitute the entry point of the Mechatron Computational Architecture. The architecture accepts information originating from one or more heterogeneous input types. Each input type acquires information independently according to its own physical, logical, or communication characteristics. The architecture does not restrict the number, type, or implementation of input types.

Examples of heterogeneous input types include, but are not limited to:

- Vision
- Audio
- Text
- Keyboard
- Touch
- Force
- Pressure
- Temperature
- Position
- Motion
- Chemical sensing
- Communication interfaces
- Internal system state
- Network information
- Future input types

These examples are illustrative only and do not limit the architecture. The architecture defines how heterogeneous input types enter the computational workflow without prescribing the technologies or engineering implementations used to acquire information from those input types.

## 8. Input Sensor Processing Units

Each heterogeneous input sensor type is associated with an independent Input Sensor Processing Unit. Multiple physical sensors of the same sensor type may be serviced by a single Input Sensor Processing Unit. Input Sensor Processing Units do not construct the Common Representation. Their responsibility is limited to sensor-specific processing and preparation of processed information suitable for architectural integration by the IIU. Standardisation of processed input does not imply that the input is sufficient or suitable for the computational objective. Information sufficiency is determined by the Central Computational Unit (CCU).

### 8.1 ISPU Architectural Rule

- One Input Sensor Processing Unit is associated with each input sensor type
- Multiple physical sensors of the same input sensor type are serviced by the same Input Sensor Processing Unit
- Physical sensors of different input sensor types are serviced by different Input Sensor Processing Units
- Therefore, if a machine supports N input sensor types, it contains N Input Sensor Processing Units, irrespective of the number of physical sensors for each input sensor type

**Example:**

- 5 cameras → 1 Visual Input Sensor Processing Unit;
- 5 microphones → 1 Audio Input Sensor Processing Unit; and
- 5 touch sensors → 1 Touch Input Sensor Processing Unit.

Therefore, five cameras, five microphones, and five touch sensors require three Input Sensor Processing Units.

An Input Sensor Processing Unit is an architectural processing component dedicated to a specific input sensor type. Each input sensor type is serviced by its own independent Input Sensor Processing Unit, which performs all processing required for information originating from one or more sensors of that type. An Input Sensor Processing Unit processes information only from its associated input sensor type and does not process information originating from other input sensor types.

Processed information is transferred to the Input Integration Unit. The Input Integration Unit converts the processed information received from each Input Sensor Processing Unit into the Common Representation. The resulting Common Representation inputs are then integrated by the Input Integration Unit. The architecture therefore separates input sensor-specific processing from technology-independent information integration. An Input Sensor Processing Unit performs all processing required to convert information originating from its associated input sensor type into a standardised form suitable for architectural integration.

Typical responsibilities include:

- Input sensor-specific preprocessing
- Calibration
- Noise reduction
- Signal interpretation
- Data validation
- Metadata generation
- Error detection
- Quality assessment

Each Input Sensor Processing Unit operates independently of every other Input Sensor Processing Unit. The architectural responsibility of each Input Sensor Processing Unit is independent of its implementation. Each Input Sensor Processing Unit shall produce processed information using standardised architectural interfaces for consumption by the Input Integration Unit, irrespective of the number of physical sensors associated with its input sensor type.

## 9. Input Integration Unit (IIU)

The Input Integration Unit (IIU) is the first primary computational component of the Mechatron Computational Architecture. The IIU converts each processed input received from the Input Sensor Processing Units (ISPUs) into the Common Representation (CR). It then integrates the resulting Common Representation inputs into the single Common Representation.

The IIU performs systematic integration activities, including:

- Information synchronisation
- Event ordering
- Concurrent event analysis
- Parallel event analysis
- Simultaneous event analysis
- Duplicate detection
- Overlap detection
- Information consolidation
- Semantic relationship analysis
- Context construction

The conversion of each processed input into the Common Representation occurs within the IIU before integration. Upon completion of integration, the single Common Representation is transmitted to the Central Computational Unit (CCU). The Common Representation is the standardised computational representation used by the remainder of the architecture. The IIU does not perform computational reasoning or determine the computational solution. Computational reasoning and computational solution determination are performed by the Central Computational Unit (CCU). The IIU operates independently of input sensor types, hardware technologies, software implementations, computational methods, and engineering implementations.

## 10. Common Representation

The Common Representation (CR) is the single standardised computational representation defined by the Mechatron Computational Architecture. All internal computational processing shall operate on the Common Representation.

The Common Representation is:

• Each processed input is converted by the IIU into the Common Representation before the resulting Common Representation inputs are integrated by the IIU into the single Common Representation
• Processed by the Central Computational Unit (CCU)
• Consumed by the Output Integration Unit (OIU)

The Common Representation is an architectural abstraction independent of its engineering realisation. The formal structure, semantics, interfaces, and engineering specification of the Common Representation are defined separately by the Common Representation specification. The Common Representation represents standardised computational information derived from the processed inputs. Information is processed by the respective Input Sensor Processing Units, and each processed input is converted by the IIU into the Common Representation before the resulting Common Representation inputs are integrated by the IIU.

**10.1 Engineering Representation**
The engineering representation of the Common Representation shall be selected according to the engineering purpose and implementation requirements.

The engineering representation shall support:
- representation of integrated computational information
- computational processing
- communication between the IIU, CCU, and OIU
- sufficient representation capacity for the computational objective
- consistent representation throughout the internal computational workflow

Binary Representation is the preferred engineering representation of the Common Representation because of its universal compatibility, compact representation, computational processing suitability, and communication suitability.

Alternative engineering representations, including hybrid, numerical, symbolic, graph-based, and future representations, may be implemented according to engineering requirements. The selected engineering representation shall be maintained throughout the internal computational workflow: IIU → CCU → OIU. The Common Representation shall not be converted into a different internal representation between these architectural components.

**10.2 Physical Communication Medium**
The physical communication medium provides the physical mechanism for transporting the selected Engineering Representation of the Common Representation between architectural components. The physical communication medium shall be selected according to the engineering purpose and implementation requirements.

Physical communication media may include:
- electrical signals
- optical/light signals
- electromagnetic radio signals
- future communication media

Optical communication is the preferred physical communication medium for transporting the selected Engineering Representation of the Common Representation. The architecture does not mandate a specific physical communication medium. The physical communication medium performs information transport only. It does not perform Common Representation conversion, Common Representation integration, computational reasoning, output integration, or output-specific processing.

**10.3 Common Representation Continuity**
The engineering representation and physical communication medium shall not alter the architectural definition of the Common Representation. The selected Common Representation shall remain consistent throughout:
IIU → CCU → OIU

The engineering representation and physical communication medium may be modified according to engineering requirements without modifying the architectural specification, provided that the Common Representation, architectural interfaces, component responsibilities, and computational workflow remain satisfied.

# 11. Central Computational Unit (CCU)

### 11.1 CCU Purpose

The Central Computational Unit (CCU) is the primary computational component of the Mechatron Computational Architecture. Its responsibility is to perform systematic computational reasoning on the Common Representation (CR) to determine the computational solution that best satisfies the current computational objective. The CCU receives the single integrated Common Representation (CR) produced by the IIU, performs computational reasoning, updates the computational content of the Common Representation, and provides the Common Representation containing the updated computational content to the Output Integration Unit. The CCU operates independently of sensing technologies, input technologies, output technologies, hardware platforms, software implementations, programming languages, computational methods, algorithms, computational frameworks, and engineering implementations. The architecture defines the responsibilities of the CCU without prescribing how those responsibilities are implemented. MCA provides computational reasoning as an architectural function and does not require consciousness as a prerequisite for performing computational reasoning or verification.

### 11.2 Principle of Computational Reasoning

The CCU performs systematic computational reasoning using the Common Representation. The Mechatron Computational Architecture requires the CCU to execute reasoning mechanisms for the current computational objective and available information. Each participating reasoning mechanism independently evaluatesthe Common Representation from its own computational perspective and may generate one or more solutions. These reasoning mechanisms may also dynamically interact during computation, with information, intermediate results, deductions, constraints, evaluations, or other computational contributions produced by one reasoning mechanism contributing to and influencing the computation performed by another. The complete set of solutions is then systematically compared and evaluated to determine the computational solution that demonstrates the highest overall consistency.

The objective of the CCU is not to determine the first possible solution, but to determine the solution that is most consistent with:

- the computational objective
- the Common Representation
- the complete computational context
- available information
- verified evidence
- engineering rules
- environmental consistency

### 11.3 CCU Computational Reasoning Process

#### 11.3.1 Receive the Common Representation

The CCU receives the single integrated Common Representation (CR) produced by the IIU.

#### 11.3.2 Read and Determine Meaning

The CCU reads the CR and determines the computational meaning of the information represented within it.

#### 11.3.3 Perform Computational Reasoning and Analysis

The CCU performs systematic computational reasoning and analysis using the computational meaning of the CR and the available computational context. The participating reasoning mechanisms operate in parallel, simultaneously, and concurrently. Each mechanism independently reasons from the determined meaning and available computational context. During this computation, the reasoning mechanisms may dynamically generate computational leads and interact through cross-contribution. Information, intermediate results, deductions, constraints, evaluations, patterns, and other computational contributions generated by one

reasoning mechanism may provide information or computational leads required by another reasoning mechanism. This interaction may continue across multiple reasoning mechanisms, including mechanisms whose relevance was not initially determined. The participating reasoning mechanisms shall continue this dynamic reasoning, lead generation, interaction, and cross-contribution toward determining a computational solution.

**11.3.4 Assess Information Sufficiency, When a Sufficiently Supported Solution Has Not Emerged**

The CCU determines whether the participating reasoning mechanisms have produced sufficient computational support for a solution that satisfies the computational objective.

- If a sufficiently supported computational solution emerges, the CCU proceeds to systematic evaluation of the solution.
- If no sufficiently supported computational solution emerges, the CCU assesses whether additional information is required to continue systematic computation. Unknown information shall never be replaced by assumptions or invented.
- If additional information is required, the CCU identifies and acquires the necessary information before continuing computation.
- If the information is already available but reasoning fails, the reasoning process requires diagnosis or troubleshooting. Information insufficiency and reasoning failure are distinct conditions.
- The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome.

**11.3.5 Evaluate Solutions**

The CCU systematically evaluates the computational solution or solutions against the computational objective and available information.

Evaluation includes comparison of solutions to identify:

- agreements
- conflicts
- inconsistencies
- complementary information
- computational strengths and limitations

Each solution is evaluated using the complete computational context, including:

- the computational objective
- the Common Representation
- computational context
- verified information
- user-provided information
- sensor observations
- engineering rules
- available evidence
- temporal consistency
- spatial consistency
- interaction consistency, where applicable
- domain-specific constraints
- confidence

Solutions that are inconsistent, contradictory, unsupported, or insufficiently supported shall be eliminated.

**11.3.6 Select the Computational Solution**

The CCU selects the computational solution that satisfies the computational objective based on systematic evaluation.

**11.3.7 Update Computational Content**

The CCU updates only the computational content of the existing CR with the computational result. The CR itself remains unchanged.

### 11.3.8 Acquire Additional Information, When Required

When additional information is required, the CCU shall identify the information necessary to continue systematic computation. The required information shall be obtained from the appropriate information source. Information sources may include users, sensors, databases, documents, network services, external computational systems, or other available information sources. If the appropriate information source is an interactive user or external agent, the CCU may generate a Bridging Question that requests only the minimum additional information necessary to continue systematic computation.

### 11.3.9 Continue Computational Reasoning Using the Same CR

Following acquisition of the required information, the computational content of the Common Representation shall be updated, while the Common Representation itself remains unchanged. The CCU shall continue computational reasoning and analysis using the same Common Representation containing the updated computational content, including parallel reasoning, dynamic lead generation, interaction, and cross-contribution among participating reasoning mechanisms.

This process shall continue until:

- the computational objective has been achieved
- a sufficiently supported computational solution has been determined
- the required information cannot be obtained

## 11.4 Computational Reasoning Mechanisms

The CCU contains multiple computational reasoning mechanisms. For every computational objective, reasoning mechanisms shall participate comprehensively in the computational reasoning process. Participation shall not be restricted solely because the relevance of a reasoning mechanism has not been initially determined. Each reasoning mechanism contributes an independent computational perspective.
Examples include, but are not limited to:

**Language Reasoning**

- Literal reasoning
- Literal meaning reasoning
- Grammar reasoning
- Typographical reasoning
- Linguistic reasoning
- Accent and dialect reasoning
- Translation and interpretation

**Logical and Analytical Reasoning**

- Rule-based reasoning
- Logical reasoning
- Analytical reasoning
- Rational reasoning
- Mathematical reasoning
- Symbolic reasoning
- Pattern reasoning
- Causal reasoning
- Analogical reasoning

**Contextual Reasoning**

- Contextual reasoning
- Interaction reasoning
- Temporal reasoning
- Spatial reasoning
- Objective-oriented reasoning

**Probabilistic Reasoning**

- Probabilistic reasoning
- Hypothetical reasoning
- Alternative-Path Reasoning

- Heuristic reasoning, where implemented by the computational system

**Specialised Reasoning**

- Domain-specific reasoning
- Engineering reasoning
- Scientific reasoning
- Medical reasoning
- Legal reasoning
- Financial reasoning
- Additional reasoning mechanisms

The architecture places no upper limit on the number of computational reasoning mechanisms. Additional reasoning mechanisms may be incorporated without modifying the architectural specification.

### 11.4.1 Comprehensive Participation of Reasoning Mechanisms

MCA does not impose a restriction that excludes a reasoning mechanism from computational participation solely because its relevance has not been initially determined. Reasoning mechanisms may participate broadly in computational reasoning, particularly for complex computational objectives, because a contribution from one reasoning mechanism may generate information, intermediate results, deductions, constraints, evaluations, patterns, or other computational leads that become relevant through dynamic interaction and cross-contribution with other reasoning mechanisms. No inherent priority, hierarchy, superiority, or fixed computational authority is assigned to any reasoning mechanism. The combined computational contributions of participating reasoning mechanisms may produce results, leads, refinements, and solutions that may not be obtainable from an isolated reasoning mechanism.

### 11.4.2 Dynamic Interaction of Reasoning Mechanisms

The participating reasoning mechanisms are not necessarily isolated from one another. They may dynamically interact during computation, with information, intermediate results, deductions, constraints, evaluations, or other computational contributions produced by one reasoning mechanism contributing to and influencing the computation performed by another. Through such dynamic cross-contribution, multiple reasoning mechanisms may collectively derive, refine, evaluate, and converge toward computational solutions in accordance with the computational objective. A reasoning mechanism whose relevance has not been initially determined may nevertheless contribute to the computation through dynamic lead generation, interaction, and cross-contribution. Such contributions may subsequently influence the computation performed by other reasoning mechanisms.

## 11.5 Interaction Pattern Learning

Interaction Pattern Learning supports systematic computational reasoning by improving the machine's understanding of interaction characteristics that influence computational context.

Examples include:

- communication styles
- interaction patterns
- response patterns
- contextual interaction characteristics
- decision-making patterns
- probabilistic interaction tendencies

Interaction models improve contextual interpretation and reasoning accuracy. The architecture does not memorise conversations or reproduce non-computational characteristics. It continuously refines interaction models that improve future computational reasoning.

## 11.6 CCU Core Responsibilities

The Central Computational Unit is responsible for:

- executing participating computational reasoning mechanisms
- generating solutions
- evaluating solutions

- comparing computational alternatives
- assessing confidence
- eliminating unsupported solutions
- selecting computational solutions
- identifying insufficient information
- acquiring additional information when required
- updating the computational content of the Common Representation with computational results

These responsibilities are defined independently of implementation technologies.

### 11.7 CCU Computational Objective

The objective of the Central Computational Unit is to determine the computational solution that is most strongly supported by systematic computation. The responsibilities of the Central Computational Unit are defined independently of algorithms, computational frameworks, mathematical techniques, hardware technologies, software implementations, and engineering methods.

## 12. Output Integration Unit (OIU)

### 12.1 OIU Purpose

The Output Integration Unit (OIU) is the primary architectural component responsible for converting the computational content of the Common Representation (CR) into standardised output representations suitable for the corresponding Output Sensor Processing Units (OSPUs). The OIU receives the same Common Representation from the Central Computational Unit (CCU), containing the computational content updated by the CCU. The OIU converts the computational content of the Common Representation into one or more output representations required to satisfy the computational objective and provides each required output representation to its corresponding Output Sensor Processing Unit. The output representations may include associated computational information, metadata, sequencing, coordination, synchronisation, prioritisation, consolidation, and routing information required for the corresponding output. The OIU performs output integration independently of individual output sensor types, hardware platforms, communication protocols, software implementations, and engineering methods. The architecture defines the responsibilities of the OIU without prescribing how those responsibilities are implemented.

### 12.2 Output Integration

The OIU converts the computational content of the Common Representation into output representations corresponding to the required heterogeneous output sensor types.

During output integration, the OIU may perform activities including, but not limited to:

- output interpretation
- output decomposition
- output coordination
- output synchronisation
- output prioritisation
- output sequencing
- output consolidation
- output routing
- output representation conversion
- metadata association

These activities prepare the computational results for processing by the corresponding Output Sensor Processing Units while preserving consistency with the computational objective. The OIU does not perform computational reasoning. The OIU receives and consumes the same single Common Representation produced through the IIU and processed by the CCU. The CCU updates only the computational content of the Common Representation. The Common Representation itself remains unchanged. The OIU converts the computational content of that same Common Representation into the required output representations.

### 12.3 Multiple Output Representations

A computational objective may require one or more heterogeneous outputs. The OIU shall convert the computational content of the Common Representation into an output representation corresponding to each required output sensor type. Each required output representation shall be provided only to its corresponding

Output Sensor Processing Unit. Multiple output representations may be produced simultaneously or sequentially according to the computational objective. Each output representation shall remain computationally consistent with the computational content of the Common Representation from which it is converted. Only the Output Sensor Processing Unit(s) corresponding to the required output sensor type(s) shall receive the respective output representations for output processing.

### 12.4 Output Coordination

When multiple output sensor types participate in the same computational objective, the OIU shall coordinate their output representations to ensure coherent system operation.

Output coordination may include:

- synchronisation of outputs
- ordering of outputs
- coordination of concurrent outputs
- dependency management
- consistency verification
- objective-oriented output organisation
- output sequencing
- output routing

The architecture specifies the responsibility for output coordination without prescribing implementation techniques.

### 12.5 OIU Core Responsibilities

The Output Integration Unit is responsible for:

- receiving the same Common Representation containing the computational content updated by the Central Computational Unit
- interpreting the computational content for output
- converting the computational content into the required output representations
- associating required computational information and metadata with the output representations
- coordinating multiple output representations
- sequencing output representations
- synchronising output representations where required
- routing each output representation to its corresponding Output Sensor Processing Unit
- maintaining consistency between output representations and the computational objective
- providing standardised architectural interfaces between computational processing and heterogeneous output sensor types

These responsibilities are architectural and remain independent of implementation technologies.

### 12.6 OIU Output Objective

The objective of the Output Integration Unit is to convert the computational content of the Common Representation into standardised output representations corresponding to the required heterogeneous output sensor types and to provide those representations to the appropriate Output Sensor Processing Unit(s) while preserving consistency with the computational objective.

## 13. Output Sensor Processing Units

### 13.1 OSPU Purpose

Each heterogeneous output sensor type is associated with an independent Output Sensor Processing Unit. Multiple physical output devices of the same output sensor type may be serviced by a single Output Sensor Processing Unit. An Output Sensor Processing Unit receives the output representation corresponding to its associated output sensor type from the Output Integration Unit and performs the processing required for that output sensor type.

### 13.2 OSPU Architectural Rule

- One Output Sensor Processing Unit is associated with each output sensor type.
- Multiple physical output devices of the same output sensor type are serviced by the same Output Sensor Processing Unit.

- Physical output devices of different output sensor types are serviced by different Output Sensor Processing Units.
- Therefore, if a machine supports N output sensor types, it contains N Output Sensor Processing Units, irrespective of the number of physical output devices for each output sensor type.

**Example:**
- 10 displays → 1 Visual Output Sensor Processing Unit
- 5 speakers → 1 Audio Output Sensor Processing Unit
- 10 robotic actuators → 1 Motion Output Sensor Processing Unit

Therefore, ten displays, five speakers, and ten robotic actuators require three Output Sensor Processing Units. The OIU provides output representations only to the Output Sensor Processing Unit(s) corresponding to the output sensor type(s) required by the computational objective. An Output Sensor Processing Unit processes information only for its associated output sensor type and does not process information for other output sensor types.

**13.3 OSPU Output Processing**

Each Output Sensor Processing Unit performs processing specific to its corresponding output sensor type.

Typical responsibilities include:
- output-specific processing
- signal generation
- command generation
- protocol adaptation
- device adaptation
- timing control
- output validation
- hardware-specific formatting
- transmission or actuation preparation

The architecture specifies these responsibilities without prescribing implementation methods.

**13.4 Standardised Architectural Interface**

Each Output Sensor Processing Unit shall receive its corresponding standardised output representation through the architectural interface provided by the Output Integration Unit. Each processing unit is responsible only for corresponding output sensor type-specific processing. The Output Sensor Processing Unit shall provide the processed output to the corresponding output sensor, actuator, or device selected according to the computational objective.

**13.5 OSPU Core Responsibilities**

Each Output Sensor Processing Unit is responsible for:
- receiving its corresponding standardised output representation
- performing sensor type-specific output processing
- preparing the corresponding output for execution
- validating output information before transmission or actuation
- delivering the processed output to the corresponding selected output sensor, actuator, or device

These responsibilities are independent of hardware technologies, communication protocols, software implementations, and engineering methods.

**13.6 OSPU Output Objective**

The objective of every Output Sensor Processing Unit is to convert its corresponding standardised output representation into a sensor type-specific output suitable for its corresponding selected output sensor, actuator, or device while preserving consistency with the computational objective.

# 14. Heterogeneous Outputs

### 14.1 Heterogeneous Outputs Purpose

Heterogeneous Outputs constitute the exit point of the Mechatron Computational Architecture. They represent the observable results of systematic computation after processing by the Output Sensor Processing Units. The architecture remains independent of the technologies used to communicate or perform outputs.

### 14.2 Output Technologies

Each heterogeneous output technology performs its assigned output using the processed information provided by its corresponding Output Sensor Processing Unit.

Examples include, but are not limited to:

- visual displays
- audio systems
- communication interfaces
- robotic manipulators
- industrial controllers
- medical devices
- printers
- vehicle control systems
- software interfaces
- future output technologies

These examples are illustrative and do not restrict the architecture.

### 14.3 Heterogeneous Outputs Objective

The objective of Heterogeneous Outputs is to communicate or perform the computational results produced by the Mechatron Computational Architecture through one or more heterogeneous output technologies while preserving consistency with the computational objective.

# 15. Output Verification

### 15.1 Output Verification Purpose

Verification is the architectural process that determines whether the computational objective has been achieved. The completion of computational processing or output generation does not, by itself, indicate successful completion of a computational objective. Verification establishes whether the observed outcome is consistent with the intended objective and provides the basis for terminating or continuing the computational workflow. The architecture defines the responsibility for verification without prescribing how verification is implemented.

### 15.2 Output Verification Process

Verification evaluates the outcome of a completed computational cycle against the defined computational objective.

Verification may use one or more independent sources of evidence, including, but not limited to:

- sensor observations
- environmental feedback
- user confirmation
- system status
- measured outcomes
- task completion
- external verification mechanisms
- future verification technologies

The architecture places no restriction on the number or type of verification mechanisms.

### 15.3 Output Verification Outcome

Verification shall determine one of the following outcomes:

- the computational objective has been achieved
- the computational objective has not been achieved
- the verification outcome cannot be determined from the available information

If the computational objective has been achieved, the computational workflow terminates. Otherwise, the computational workflow continues, and additional information shall be acquired when required.

### 15.4 Output Verification Objective

The objective of verification is to determine, through evidence, whether the computational objective has been achieved and to provide the architectural basis for terminating or continuing the computational workflow.

## 16. Computational Workflow Iteration

### 16.1 Workflow Iteration Purpose

The Mechatron Computational Architecture operates as an iterative computational system. A computational objective may require multiple computational cycles before it can be successfully achieved. Each computational cycle begins with the Common Representation containing the computational content of the current computational state and the most recently available information.

### 16.2 Iterative Computation

Following verification, the architecture determines whether an additional computational cycle is required.
If verification confirms that the computational objective has been achieved, the computational workflow terminates. If verification determines that the computational objective has not been achieved, the architecture initiates a new computational cycle using updated information obtained from the environment, users, or connected systems. Each new computational cycle shall remain consistent with the current computational objective.

### 16.3 Evidence-Based Iteration

Every computational cycle shall operate using verified and currently available information. Unknown information shall never be replaced by assumptions. When additional information becomes available, it shall be incorporated into the Common Representation before the next computational cycle begins. The architecture therefore improves computational outcomes through systematic evidence acquisition and verification.

### 16.4 Workflow Continuity

The computational workflow shall continue until one of the following conditions is satisfied:

- the computational objective has been achieved
- the computational objective has been terminated
- the required information cannot be obtained

The architecture does not prescribe the number of computational cycles required to satisfy a computational objective.

### 16.5 Workflow Iteration Objective

The objective of computational workflow iteration is to enable systematic, evidence-based computation through repeated computational cycles until the computational objective has been satisfied.

## 17. Continuous Computational Improvement

### 17.1 Continuous Improvement Purpose

Continuous Computational Improvement is an architectural characteristic of the Mechatron Computational Architecture. Its purpose is to improve computational capability throughout the operational lifetime of the computational machine while preserving the stability of the architectural specification. The architecture improves computational capability through refinement of computational methods, computational knowledge, and engineering procedures without memorisation of previous questions, conversations, or answers. The architecture defines the responsibility for continuous computational improvement without prescribing how improvement is implemented.

**17.2 Principles of Continuous Improvement**
Continuous Computational Improvement shall:

- improve computational capability through verified computational outcomes
- preserve the architectural principles defined by the Mechatron Computational Architecture
- remain independent of implementation technologies
- operate without modifying the architectural structure
- support increasingly accurate, reliable, and efficient systematic computation

The architecture improves computational operation without modifying the architecture.

**17.3 Scope of Improvement**
Continuous Computational Improvement may refine, where applicable:

- computational procedures
- computational logic
- computational reasoning mechanisms
- engineering rules
- domain knowledge
- interaction models
- contextual interpretation
- pattern recognition
- solution evaluation
- confidence estimation
- solution selection quality
- computational performance

The architecture places no restriction on additional forms of computational improvement.

**17.4 Verified Improvement**
Continuous Computational Improvement shall be based upon verified computational outcomes. Improvements shall not be derived from unsupported assumptions, unverified information, or speculative conclusions. Verification provides the architectural basis for determining whether computational improvements are justified.

**17.5 Continuous Improvement Objective**
The objective of Continuous Computational Improvement is to continuously enhance computational capability while preserving the architectural stability and systematic operation of the Mechatron Computational Architecture.

## 18. Technology Independence

The Mechatron Computational Architecture specifies the computational architecture without prescribing the technologies used to implement its architectural components, responsibilities, interfaces, relationships, or computational workflow.

The architecture does not prescribe:
• hardware technologies
• sensing technologies
• input technologies
• output technologies
• storage technologies
• programming languages
• operating systems
• software frameworks
• computational methods
• algorithms
• artificial intelligence models
• communication technologies
• engineering implementations

The engineering representation of the Common Representation and the physical communication medium used to transport the selected Engineering Representation are implementation choices and are not prescribed by the architectural specification. Changes to implementation technologies do not require modification of the architectural specification, provided that the defined architectural components, responsibilities, interfaces, relationships, and computational workflow remain satisfied.

## 19. Architectural Independence

The Mechatron Computational Architecture specifies the organisation and computational relationships of the machine independently of specific engineering implementations.

The architectural specification defines:

- architectural components
- component responsibilities
- architectural interfaces
- component relationships
- computational workflow
- Common Representation–based computation
- architectural structure

These architectural definitions remain applicable when the engineering implementation of the corresponding components, interfaces, computational methods, or physical systems changes. Architectural Independence therefore preserves the defined architectural organisation while permitting implementation details to be specified and modified at the engineering level.

## 20. Summary

The Mechatron Computational Architecture (MCA) defines a systematic, technology-independent computational architecture for computational machines. The architecture establishes a complete computational workflow in which heterogeneous information is processed through specialised architectural components, each processed input is converted into the Common Representation within the Input Integration Unit, the resulting Common Representation inputs are integrated into a single Common Representation, and the integrated Common Representation is systematically evaluated to satisfy computational objectives before being transformed into heterogeneous outputs and verified against the intended objective. By separating computational responsibilities into independent architectural components, the MCA provides a stable architectural specification that supports extensibility, interoperability, maintainability, and implementation across diverse engineering domains. The architecture specifies what each architectural component is responsible for while remaining independent of how those responsibilities are implemented. This separation enables computational methods, engineering techniques, hardware technologies, software systems, and future implementations to evolve without altering the architectural principles of the Mechatron Computational Architecture.

# Part II
# Common Representation

## 1. Purpose

The Common Representation is the standardised internal computational representation of the Mechatron Computational Architecture. It provides a common computational interface through which information is exchanged between the Input Integration Unit (IIU), Central Computational Unit (CCU), and Output Integration Unit (OIU). The Common Representation enables computational processing independently of sensor technologies, input technologies, output technologies, communication protocols, hardware platforms, or implementation methods.

The Common Representation specification defines:

- the Common Representation as an architectural concept
- its lifecycle
- engineering representation
- physical communication medium
- representation integrity
- engineering selection principles

## 2. Design Principles

The Common Representation is governed by the following architectural principles:

- A single standardised internal representation shall be used throughout the computational architecture
- The Common Representation shall remain independent of implementation technologies
- All computational processing within the architecture shall operate upon the Common Representation
- The Common Representation defines the computational interface between architectural modules independently of its engineering implementation
- The architecture specifies the computational representation while allowing implementation flexibility
- Single Common Representation
- Representation Consistency
- Engineering Representation Independence
- Physical Communication Medium Independence
- Technology Independence
- Engineering Flexibility
- Extensibility

## 3. Common Representation Lifecycle

The Common Representation progresses through a defined architectural lifecycle.

**i.** The Input Integration Unit (IIU) converts each processed input received from the Input Sensor Processing Units (ISPUs) into its corresponding Common Representation ($CR_i$). It then integrates the resulting representations into a single Common Representation (CR).
**ii.** The Central Computational Unit (CCU) receives the single integrated Common Representation (CR) from the IIU and performs the following computational process:

- Receive the Common Representation.
- Read and determine its meaning.
- Perform computational reasoning and analysis using the determined meaning and available computational context.
- Continue parallel reasoning, dynamic lead generation, interaction, and cross-contribution until a sufficiently supported computational solution emerges or no such solution emerges.
- Assess information sufficiency when no sufficiently supported computational solution emerges.
- Acquire additional information only when required.
- If information is available but reasoning fails, diagnose or troubleshoot the reasoning failure.

- Evaluate the computational solution or solutions against the computational objective and available information when a sufficiently supported solution emerges.
- Select the computational solution that satisfies the computational objective.
- Update only the computational content of the same CR.
- Produce the updated Common Representation (CR').

The Common Representation itself remains unchanged throughout the computational process. Only its computational content is updated.

**Note:** The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome.

**iii.** The Output Integration Unit (OIU) receives the same Common Representation (CR) containing the updated computational content from the Central Computational Unit (CCU). The OIU converts the computational content of the Common Representation into the required OSPU-specific Output Representations, including associated metadata, sequencing, coordination, synchronisation, prioritisation, and routing information.

**iv.** The corresponding Output Sensor Processing Unit(s) (OSPUs) receive their respective OSPU-specific Output Representations and perform output sensor type-specific processing. The resulting output is provided to the selected corresponding output sensor, actuator, or device for execution.

This lifecycle defines the movement and use of the Common Representation throughout the computational architecture.

## 4. Representation Invariance

The Common Representation remains the same standardised internal computational representation throughout the computational workflow. No architectural component creates an alternative internal representation during computational processing. The Common Representation is maintained from its conversion and integration within the Input Integration Unit (IIU), through computational reasoning within the Central Computational Unit (CCU), to its consumption by the Output Integration Unit (OIU). The Central Computational Unit (CCU) updates only the computational content of the Common Representation. The Common Representation itself remains unchanged. The Output Integration Unit (OIU) receives the same Common Representation containing the updated computational content and converts its computational content into the required OSPU-specific Output Representations. The corresponding Output Sensor Processing Unit(s) (OSPUs) process their respective OSPU-specific Output Representations for their associated output sensor types and provide the resulting output to the selected corresponding output sensor, actuator, or device.

The following remain invariant throughout IIU → CCU → OIU:

- one Common Representation
- one engineering representation
- architectural semantics
- representation integrity

Only the computational content of the Common Representation changes throughout the computational process.

## 5. Common Representation Structure

### 5.1 Purpose

The Common Representation provides the standardised internal computational representation used throughout the Mechatron Computational Architecture. It enables information to be exchanged between the Input Integration Unit (IIU), Central Computational Unit (CCU), and Output Integration Unit (OIU) through a common computational interface while remaining independent of implementation technologies.

### 5.2 Standardised Representation

The Common Representation shall maintain a standardised representation throughout the internal computational workflow. The architecture specifies the computational interface of the representation without prescribing its engineering encoding or implementation. Different implementation technologies may employ different internal encodings while preserving the same architectural representation.

**5.3 Computational Content**
The computational content of the Common Representation is progressively updated throughout computation. The Input Integration Unit (IIU) converts each processed input into the Common Representation (CR). It then integrates the resulting Common Representation inputs into the single Common Representation. The Central Computational Unit (CCU) performs computational reasoning by updating the computational content of the Common Representation. The standardised representation remains unchanged while its computational content is progressively updated throughout the computational workflow.

**5.4 Standardised Metadata**
The Common Representation supports standardised metadata generated during sensor processing and preserved for computational processing. The architecture specifies standardised metadata while remaining independent of the engineering implementation of that metadata.

**5.5 Technology Independence**
The Common Representation is independent of sensor technologies, input technologies, output technologies, hardware platforms, software implementations, communication protocols, and engineering data structures. Changes to implementation technologies do not require modification of the Common Representation or the computational architecture.

**5.6 Engineering Representation**
The architecture does not prescribe or require any single specific type of engineering representation for the Common Representation. The Engineering Representation is the engineering realisation of the Common Representation selected for implementation. It defines how the Common Representation is represented internally for storage, communication, and computational processing without altering its architectural definition, computational meaning, interfaces, or computational properties. The engineering representation shall be selected according to the engineering purpose and implementation requirements of the computational system.

Possible engineering representations include, but are not limited to:

- Binary representations
- Hybrid representations
- Numerical representations
- Symbolic representations
- Graph representations
- Semantic representations
- Structured command representations
- Encodings
- Embeddings
- Future engineering representations

Binary Representation is the preferred engineering representation because of its universal compatibility, compact representation, memory efficiency, computational efficiency, and suitability for communication. However, the architecture does not mandate Binary Representation. Alternative engineering representations, including hybrid and future engineering representations, may be selected where appropriate according to the engineering purpose and implementation requirements. Irrespective of the engineering representation selected, the architectural definition, computational meaning, interfaces, and computational properties of the Common Representation remain unchanged.

**5.7 Physical Communication Medium**
The architecture does not prescribe or require any single physical communication medium for transporting the selected Engineering Representation of the Common Representation. The Physical Communication Medium is the engineering medium used to communicate the selected engineering representation of the Common Representation between architectural components. The physical communication medium shall be selected according to the engineering purpose and implementation requirements of the computational system.

Examples of physical communication media include, but are not limited to:

- Electrical signals

- Optical (light) communication
- Electromagnetic radio communication
- Future communication media

Optical (light) communication is the preferred physical communication medium for transporting the selected Engineering Representation of the Common Representation because of its high bandwidth, high communication speed, and suitability for transporting the selected Engineering Representation of the Common Representation. However, the architecture does not mandate any specific physical communication medium. Alternative communication media may be selected where required by the engineering purpose and implementation. Irrespective of the physical communication medium selected, the architectural definition, computational meaning, interfaces, and computational properties of the Common Representation remain unchanged.

### 5.8 Representation Selection Principles

The selection of the engineering representation and physical communication medium shall be based on the engineering purpose and implementation requirements of the computational system.

Selection may consider factors including, but not limited to:

- Engineering purpose
- Implementation requirements
- Computational efficiency
- Memory efficiency
- Communication efficiency
- Scalability
- Future extensibility

The selected engineering representation and physical communication medium shall provide efficient and reliable realisation of the Common Representation while preserving the architectural definition, computational meaning, interfaces, and computational properties defined by the Mechatron Computational Architecture. Once selected, the engineering representation shall be used consistently throughout the internal computational workflow from the Input Integration Unit (IIU), through the Central Computational Unit (CCU), to the Output Integration Unit (OIU). Similarly, the selected physical communication medium shall be used consistently throughout the internal computational workflow unless an alternative engineering implementation is explicitly required. Irrespective of the engineering representation or physical communication medium selected, the architectural specification of the Common Representation remains unchanged.

### 5.9 Extensibility

The Common Representation is designed to remain extensible. Future implementation technologies and engineering representation types may be incorporated without modifying the standardised computational representation or the architectural principles of the Mechatron Computational Architecture.

## 6. Common Representation Operations

### 6.1 Purpose

The Common Representation undergoes a defined sequence of operations throughout the computational workflow of the Mechatron Computational Architecture. These operations specify how processed inputs are converted into the Common Representation, integrated, processed through computational reasoning, updated, transmitted, and consumed while maintaining a single standardised internal representation.

### 6.2 Conversion and Integration (IIU)

The Input Integration Unit (IIU) converts each processed input received from the Input Sensor Processing Units (ISPUs) into the Common Representation (CR). It then integrates the resulting Common Representation inputs into the single Common Representation. Upon completion of integration, the single Common Representation is transmitted to the Central Computational Unit (CCU).

### 6.3 CCU Computational Reasoning Process

The Central Computational Unit (CCU) receives the single integrated Common Representation (CR) produced by the Input Integration Unit (IIU) and performs the following computational process.

**Receive the Common Representation**
The CCU receives the single integrated Common Representation (CR) produced by the IIU.

**Read and Determine Meaning**
The CCU reads the CR and determines the computational meaning of the information represented within it.

**Perform Computational Reasoning and Analysis**
The CCU performs systematic computational reasoning and analysis using the determined computational meaning and available computational context. Participating reasoning mechanisms operate in parallel, simultaneously, and concurrently. During computation, they dynamically generate computational leads, interact, and cross-contribute. Contributions from one mechanism may provide information or computational leads required by another, including a mechanism whose relevance was not initially determined.

**Assess Information Sufficiency**
If no sufficiently supported computational solution emerges through reasoning, dynamic lead generation, interaction, and cross-contribution, the CCU assesses information sufficiency. Unknown information shall never be replaced by assumptions or invented. Additional information shall be acquired only when required to continue systematic computation. If the information is already available but reasoning fails, the reasoning process requires diagnosis or troubleshooting. Information insufficiency and reasoning failure are distinct conditions.

**Evaluate Solutions**
When a sufficiently supported computational solution emerges, the CCU systematically evaluates the solution or solutions against the computational objective and available information.

**Select the Computational Solution**
The CCU selects the computational solution that satisfies the computational objective based on systematic evaluation.

**Update Computational Content and Continue Computation**
The CCU updates only the computational content of the existing CR; the Common Representation itself remains unchanged. When additional information is required, the CCU incorporates it into the computational content and continues reasoning using the same CR.

The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome.

**6.4 Computational Content Update and Continued Computation**
The CCU updates only the computational content of the existing Common Representation (CR); the CR itself remains unchanged. When additional information is required, the CCU shall identify and acquire it from the appropriate information source. If the information must be obtained from an interactive user or external agent, the CCU may generate a Bridging Question requesting only the minimum additional information necessary to continue systematic computation. The CCU incorporates the acquired information into the computational content of the existing CR. The CR itself remains unchanged. The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome.

**6.5 Transmission**
The Common Representation is transmitted between architectural components without changing its standardised representation. The architecture specifies the computational data flow. The communication mechanisms used for transmission are implementation dependent. Transmission uses the selected physical communication medium to transport the selected Engineering Representation of the Common Representation. Representation remains unchanged.

### 6.6 Consumption (OIU)

The Output Integration Unit (OIU) consumes the same Common Representation containing the updated computational content from the Central Computational Unit (CCU). The OIU converts the computational content of the Common Representation into the required OSPU-specific Output Representations and provides the respective representations to the corresponding Output Sensor Processing Unit(s) (OSPUs). The OIU performs the required output interpretation, coordination, synchronisation, prioritisation, sequencing, metadata association, and routing associated with the output representations. The OIU does not perform computational reasoning.

## 7. Representation Integrity

### 7.1 — Standardisation

The Common Representation shall be the single standardised internal computational representation of the Mechatron Computational Architecture. All architectural modules that operate on the Common Representation shall interpret and process it according to the same standardised computational specification. The Common Representation shall remain unchanged throughout the computational workflow. The Central Computational Unit (CCU) updates only the computational content of the Common Representation. The Output Integration Unit (OIU) receives the same Common Representation containing the updated computational content and converts that computational content into the required OSPU-specific Output Representations. The Common Representation therefore remains the single standardised internal computational representation from the Input Integration Unit (IIU), through the Central Computational Unit (CCU), to the Output Integration Unit (OIU).

### 7.2 Technology Independence

The Common Representation shall remain independent of implementation technologies.

The architecture shall not depend upon:

- Sensor technologies
- Input technologies
- Output technologies
- Communication protocols
- Programming languages
- Operating systems
- Hardware architectures
- Computational frameworks
- Engineering data structures
- Engineering Representation Independence
- Physical Communication Medium Independence

Improvements to implementation technologies shall not require modification of the Common Representation or the computational architecture.

### 7.3 No Representation Conversion within the CCU

The Central Computational Unit (CCU) shall perform computational reasoning directly upon the Common Representation. During computation, the CCU may update the computational content of the Common Representation according to the computational result. The Common Representation itself shall remain unchanged. The CCU shall not convert the Common Representation into another internal representation. The Common Representation shall continue through the IIU → CCU → OIU architectural path as the same Common Representation, with only its computational content being updated. Conversion of the computational content of the same Common Representation into the required OSPU-specific Output Representations shall occur only within the Output Integration Unit (OIU).

### 7.4 Conversion to OSPU-Specific Output Representations Only within the OIU

The Output Integration Unit (OIU) is the only architectural component responsible for converting the computational content of the Common Representation into OSPU-specific Output Representations. The OIU converts the computational content according to the required output sensor types and provides each resulting Output Representation to its corresponding Output Sensor Processing Unit (OSPU). The OIU may associate metadata, sequencing, coordination, synchronisation, prioritisation, and routing information required for the

corresponding output. No other architectural component shall perform this conversion. The Common Representation remains the same throughout the IIU → CCU → OIU computational path. Only its computational content is updated by the CCU; the OIU converts that computational content into the required OSPU-specific Output Representations.

## 8. Architectural Hierarchy

Architecture
↓
Common Representation
↓
Engineering Representation
*(Binary preferred; others permitted)*
↓
Physical Communication Medium
*(Optical (light) preferred; others permitted)*
↓
Implementation
*(Hardware • Software • Physical Embodiment)*

The Architecture defines the computational organisation, responsibilities, interfaces, and workflow independently of engineering implementation. The Common Representation defines the single standardised architectural representation used throughout the internal computational workflow from the Input Integration Unit (IIU), through the Central Computational Unit (CCU), to the Output Integration Unit (OIU). The Common Representation remains the same throughout the IIU → CCU → OIU computational path. The Central Computational Unit updates only the computational content of the Common Representation. The Output Integration Unit receives the same Common Representation containing the updated computational content and converts that computational content into the required OSPU-specific Output Representations. The Engineering Representation defines the engineering realisation of the Common Representation. Binary Representation is the preferred engineering representation; however, alternative engineering representations may be selected according to the engineering purpose and implementation requirements. The Physical Communication Medium defines the engineering medium used to transport the selected engineering representation. The Implementation defines the realisation of the architecture using hardware, software, communication technologies, and physical embodiments appropriate to the engineering application. This hierarchy establishes the separation between the architectural specification, the Common Representation, and their engineering realisation. The Architecture and Common Representation remain defined by the architectural specification, while the engineering representation, physical communication medium, and implementation may be selected according to engineering requirements without modifying the architectural definition.

## 9. Summary

The Common Representation specification defines the standardised Common Representation used throughout the computational workflow of the Mechatron Computational Architecture. The Input Integration Unit (IIU) converts processed inputs into the Common Representation and integrates them into the single Common Representation. The Central Computational Unit (CCU) performs computational reasoning using the Common Representation and updates only its computational content while the Common Representation itself remains unchanged. The Output Integration Unit (OIU) receives the same Common Representation containing the updated computational content and converts its computational content into the required OSPU-specific Output Representations. The corresponding Output Sensor Processing Units (OSPUs) process their respective Output Representations for their associated output sensor types, and the resulting outputs are provided to the corresponding output sensors, actuators, or devices. The Common Representation therefore remains the single standardised internal computational representation throughout the IIU → CCU → OIU workflow, while the computational content changes according to the computational process.

# Part III
# Mathematical Model

ISPU → IIU → CCU → OIU → OSPU → Output Verification

**1. Input Sensor Processing Unit (ISPU)**

$$I_i = \{I_{i1}, I_{i2}, \ldots, I_{imi}\}, \quad i = 1, \ldots, n$$
$$m_i = |I_i|$$
$$I = \{I_1, I_2, \ldots, I_n\}$$
$$I_i \rightarrow ISPU_i$$
$$ISPU_i(I_i) = P_i$$
$$\{I_1, \ldots, I_n\} \rightarrow \{ISPU_1, \ldots, ISPU_n\} \rightarrow \{P_1, \ldots, P_n\}$$

Definitions:

- $I_i$ = incoming information for input type i.
- $I_{ij}$ = individual information item j within input type i.
- $m_i$ = number of information items in $I_i$.
- i = identifies the input type/category being considered.
- n = number of input types.
- $ISPU_i$ = Input Sensor Processing Unit associated with input type i.
- $P_i$ = processed information produced by $ISPU_i$.
- Note: Each input type is processed by its corresponding ISPU.

**2. Input Integration Unit (IIU)**

$$IIU_i(P_i) = CR_i$$
$$IIU(\{CR_1, \ldots, CR_n\}) = CR$$
$$\{P_1, \ldots, P_n\} \rightarrow \{CR_1, \ldots, CR_n\} \rightarrow CR$$

Definitions:

- $P_i$ = processed information received from $ISPU_i$.
- $CR_i$ = Common Representation corresponding to $P_i$.
- CR = single integrated Common Representation produced by the IIU.
- Note: The IIU converts the processed inputs into the Common Representation and integrates them into the single Common Representation.

**3. Central Computational Unit (CCU)**

$$CCU(CR) = CR'$$
$$CR \rightarrow M \rightarrow \{R_1, R_2, \ldots, R_k\}_{\|} \rightleftarrows L \rightarrow \begin{cases} A \rightarrow CR', & \text{if no supported solution emerges} \\ S \rightarrow E \rightarrow S^* \rightarrow CR', & \text{if a supported solution emerges} \end{cases}$$

Definitions:

- **CR:** Incoming Common Representation.
- **M:** Computational meaning determined from CR.
- **$\{R_1, R_2, \ldots, R_k\}\|$:** Participating reasoning mechanisms operating in parallel, simultaneously, and concurrently.
- **L:** Computational leads generated through dynamic reasoning, interaction, and cross-contribution.
- **A:** Information sufficiency assessment. Additional information is acquired only when required. If information is available but reasoning fails, diagnosis or troubleshooting is needed.
- **S:** A sufficiently supported computational solution.
- **E:** Systematic evaluation of the solution against the objective and available information.
- **S*:** Selected computational solution satisfying the objective.

- **CR':** The same CR with its computational content updated by the CCU.

**Note:** The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome.

**Dynamic lead generation, interaction, and cross-contribution:**
The reasoning mechanisms may dynamically generate computational leads and interact through cross-contribution. Information, intermediate results, deductions, constraints, evaluations, patterns, or other computational contributions produced by one mechanism may provide information or computational leads required by another mechanism, including a mechanism whose relevance was not initially determined.

**Information sufficiency:**
The CCU assesses information sufficiency only when a sufficiently supported computational solution has not emerged through the preceding reasoning, lead generation, interaction, and cross-contribution. If additional information is required, the CCU obtains it and continues computation using the same CR with updated computational content. Unknown information shall never be replaced by assumptions or invented. The Common Representation itself remains unchanged.

**4. Output Integration Unit (OIU)**

$$\mathrm{OIU}(\mathrm{CR}') = \{\mathrm{OR}_1, \mathrm{OR}_2, \ldots, \mathrm{OR}_n\}$$
$$\mathrm{CR}' \rightarrow \{\mathrm{OR}_1, \mathrm{OR}_2, \ldots, \mathrm{OR}_n\} \rightarrow \{\mathrm{OSPU}_1, \ldots, \mathrm{OSPU}_n\}$$

Definitions:
- CR' = Common Representation containing the computational content updated by the CCU.
- $\mathrm{OR}_i$ = OSPU-specific Output Representation for required output sensor type i.
- $\mathrm{OSPU}_i$ = Output Sensor Processing Unit associated with output sensor type i.
- Only the required output representations are routed to their corresponding OSPUs.
- Note: The OIU converts the computational content into the required OSPU-specific Output Representations and performs the required metadata association, sequencing, coordination, synchronization, prioritization, and routing.

**5. Output Sensor Processing Unit (OSPU)**

$$\mathrm{OR}_i \rightarrow \mathrm{OSPU}_i \rightarrow \mathrm{O}_i$$
$$\mathrm{OSPU}_i(\mathrm{OR}_i) = \mathrm{O}_i$$

Definitions:
- $\mathrm{OR}_i$ = OSPU-specific Output Representation received from the OIU.
- $\mathrm{OSPU}_i$ = Output Sensor Processing Unit associated with output sensor type i.
- $\mathrm{O}_i$ = processed output provided to the selected corresponding output sensor, actuator, or device for execution.
- Note: Only the required OSPU receives and processes its corresponding output representation. The OSPU performs output sensor type-specific processing.

**6. Output Verification and Iteration**

$$\mathrm{V}(\{\mathrm{O}_i\}, \mathrm{G}) = \mathrm{v}$$
$$\mathrm{v} = 1 \rightarrow \mathrm{END}$$
$$\mathrm{v} = 0 \rightarrow \text{acquire additional information} \rightarrow \text{continue computation}$$

Definitions:
- V = output verification process.
- $\{\mathrm{O}_i\}$ = processed outputs produced for the computational cycle.

- G = computational objective.
- v = verification outcome indicating whether the computational objective has been achieved.
- Note: The verification outcome determines whether the computational workflow terminates or continues. If the objective has not been achieved, additional information may be acquired and systematic computation continues through the same architecture.

**7. Complete Computational Workflow Model**

$$\{I_i\} \to \{ISPU_i\} \to \{P_i\} \to IIU \to \{CR_i\} \to CR \to CCU \to CR' \to OIU \to \{OR_i\} \to \{OSPU_i\} \to \{O_i\} \to \text{Verification}$$
$$\text{Verification: } v = 1 \to \text{END}; \quad v = 0 \to \text{additional information} \to \text{next computational cycle}$$

Unit equations:

$$ISPU_i(I_i) = P_i$$
$$IIU_i(P_i) = CR_i$$
$$IIU(\{CR_i\}) = CR$$
$$CCU(CR) = CR'$$
$$CR \to M \to \{R_1, R_2, \dots, R_k\}_{\|} \rightleftarrows L \to \begin{cases} A \to CR', & \text{if no supported solution emerges} \\ S \to E \to S^* \to CR', & \text{if a supported solution emerges} \end{cases}$$
$$OIU(CR') = \{OR_i\}$$
$$OSPU_i(OR_i) = O_i$$
$$V(\{O_i\}, G) = v$$

This complete formulation represents the mathematical flow from heterogeneous input information through input processing and integration, computational processing including parallel reasoning, dynamic lead generation, interaction, cross-contribution, and information sufficiency assessment, output integration, output-specific processing, and output verification, with continuation through additional information acquisition when the computational objective has not been achieved.

# Part IV
# Algorithms

ISPU → IIU → CCU → OIU → OSPU → Output Verification

**1. Input Sensor Processing Unit (ISPU)**

Input: $I_i$

Algorithm steps:

1. Receive information from input sensor type i.
2. Process the information according to the associated input sensor type.
3. Produce processed information $P_i$.
4. Send $P_i$ to the IIU.

$$I_i \rightarrow ISPU_i \rightarrow P_i$$

**2. Input Integration Unit (IIU)**

Input: $\{P_1, \ldots, P_n\}$

Algorithm steps:

5. Receive each $P_i$ from its corresponding ISPU.
6. Convert each $P_i$ into $CR_i$.
7. Integrate the $CR_i$ into the single Common Representation CR.
8. Provide CR to the CCU.

$$\{P_i\} \rightarrow \{CR_i\} \rightarrow CR$$

**3. Central Computational Unit (CCU)**

Input: CR

Algorithm steps:

9. Receive CR.
10. Read and determine its meaning.
11. Perform computational reasoning and analysis using the determined computational meaning and available computational context.
12. Participating reasoning mechanisms operate in parallel, simultaneously, and concurrently. During computation, they dynamically generate computational leads, interact, and cross-contribute. Contributions from one reasoning mechanism may provide information or computational leads required by another, including a mechanism whose relevance was not initially determined.
13. Continue parallel reasoning, dynamic lead generation, interaction, and cross-contribution until a sufficiently supported computational solution emerges or no such solution emerges from the reasoning process.
14. If no sufficiently supported computational solution emerges, assess information sufficiency. Additional information is acquired only when required. If information is available but reasoning fails, diagnosis or troubleshooting is needed.
15. If additional information is required, identify the information necessary to continue systematic computation and acquire it through the appropriate information source.
16. When additional information is acquired, update only the computational content of the same CR; the Common Representation itself remains unchanged.
17. When a sufficiently supported computational solution emerges, systematically evaluate the solution or solutions against the computational objective and available information.
18. Select the computational solution that satisfies the computational objective and update the computational content of the same CR.
19. Produce CR'.

$$CR \rightarrow M \rightarrow \{R_1, R_2, \ldots, R_k\}_{\|} \rightleftarrows L \rightarrow \begin{cases} A \rightarrow CR', & \text{if no supported solution emerges} \\ S \rightarrow E \rightarrow S^* \rightarrow CR', & \text{if a supported solution emerges} \end{cases}$$

Note: The same architectural route is maintained irrespective of information sufficiency; the computational content generated by the CCU determines the outcome.

**4. Output Integration Unit (OIU)**
Input: CR'
Algorithm steps:
20. Receive the same Common Representation containing updated computational content.
21. Interpret the computational content for output.
22. Convert the computational content into the required OSPU-specific Output Representations.
23. Associate required metadata.
24. Coordinate, sequence, synchronize, and prioritize outputs where required.
25. Route each required Output Representation to its corresponding OSPU.
26. Do not route representations to OSPUs that are not required.

$$CR' \rightarrow \{OR_i\}\text{required} \rightarrow \{OSPU_i\}\text{required}$$

**5. Output Sensor Processing Unit (OSPU)**
Input: $OR_i$
Algorithm steps:
27. Receive the corresponding OSPU-specific Output Representation.
28. Perform output sensor type-specific processing.
29. Prepare and validate the output for execution.
30. Provide the processed output only to the selected corresponding output sensor, actuator, or device.
31. The selected device executes the output.

$$OR_i \rightarrow OSPU_i \rightarrow O_i$$

**6. Output Verification and Iteration**
Input: Processed output $O_i$ and computational objective
Algorithm steps:
32. Verify whether the computational objective has been achieved.
33. If the computational objective has been achieved, terminate the computational workflow.
34. If the computational objective has not been achieved, acquire additional information as required.
35. Continue systematic computation through the same architectural workflow.

$$\text{Verification} \rightarrow \text{Objective achieved} \rightarrow \text{END}$$
$$\text{Verification} \rightarrow \text{Objective not achieved} \rightarrow \text{Additional information} \rightarrow \text{Continue computation}$$

**7. Complete Algorithmic Flow**

$$\{I_i\} \rightarrow \{P_i\} \rightarrow \{CR_i\} \rightarrow CR \rightarrow CCU \rightarrow CR' \rightarrow \{OR_i\}\text{required} \rightarrow \{O_i\}\text{selected} \rightarrow \text{Verification}$$
$$\text{Objective achieved} \rightarrow \text{END}$$
$$\text{Objective not achieved} \rightarrow \text{Additional information} \rightarrow \text{Next computational cycle}$$

This unit-wise algorithmic formulation follows the mathematical models and architectural sequence established for the ISPU, IIU, CCU, OIU, and OSPU, with information sufficiency assessment within the CCU and output verification at the end of the computational cycle.